AI/ML Engineering Assignment — Financial Transaction Intelligence System

Duration: 2–3 days (approx. 16–20 focused hours) **Level:** Mid to Senior AI/ML Engineer **Domain:** Finance / Audit / Fraud Analytics

---

1. Objective

Build a small but production-minded system called **"TxnIntel"** that ingests financial transaction data, detects anomalous/fraudulent transactions using ML, and generates plain-English explanations for flagged transactions using an LLM — the kind of explainability an auditor or finance reviewer would need.

This assignment tests four things: 1. Data handling and EDA discipline on messy, imbalanced financial data 2. ML modeling judgment (not just accuracy chasing) 3. LLM integration with structured, controlled outputs 4. Engineering hygiene — API design, reproducibility, documentation

---

2. Dataset

Use **any one** of the following public datasets:

- **Credit Card Fraud Detection** (Kaggle — ULB dataset, ~284K transactions, 0.17% fraud)
- **PaySim Synthetic Mobile Money Transactions** (Kaggle, ~6M rows — you may sample down to 500K–1M)
- **IEEE-CIS Fraud Detection** (Kaggle — harder, richer features; only if you want a challenge)

If you cannot access Kaggle, you may generate a synthetic transaction dataset (min. 100K rows) with a documented generation script — but real data is preferred.

---

3. Tasks

Part A — Data Engineering & EDA (Day 1, ~4 hrs)

1. Load the dataset into **PostgreSQL** (or SQLite if Postgres setup is a blocker — mention why).
2. Write a short EDA notebook/report covering:
3. Class imbalance ratio and what it implies for modeling
4. Feature distributions for fraud vs. non-fraud (pick the 5 most discriminative features)
5. Data quality issues found (nulls, duplicates, leakage risks) and how you handled them
6. Build a reproducible preprocessing pipeline (sklearn `Pipeline` or equivalent) — **no manual one-off transformations in notebooks that can't be re-run.**

Part B — ML Model: Fraud/Anomaly Detection (Day 1–2, ~6 hrs)

1. Train **two models** and compare:
2. One baseline (e.g., Logistic Regression)
3. One stronger model (e.g., XGBoost / LightGBM / Isolation Forest — your choice, justify it)
4. Handle class imbalance deliberately (class weights, SMOTE, threshold tuning — explain your choice and its trade-offs).
5. Evaluate with the **right metrics for imbalanced fraud data**:
6. Precision, Recall, F1 at your chosen threshold
7. PR-AUC (not just ROC-AUC — explain why PR-AUC matters here)
8. A cost-based view: assume a false negative costs $500 and a false positive costs $25 (analyst review time). Pick your operating threshold based on this, and show the math.
9. Produce **feature importance / SHAP values** for the final model — these feed Part C.
10. **Reproducibility requirement:** fix all random seeds, log model version + training data hash, and save the model artifact. Anyone should be able to retrain and get the same numbers.

Part C — LLM Explanation Layer (Day 2, ~4 hrs)

For each flagged transaction, generate a **structured audit-style explanation** using an LLM (Claude API, OpenAI API, or a local model via Ollama — your choice; if no API key access, mock the LLM call behind an interface and show the prompt design).

Requirements: 1. Input to the LLM: transaction details + model score + top 3–5 SHAP/feature contributions. 2. Output must be **strict JSON** with this schema: `json { "risk_level": "HIGH | MEDIUM | LOW", "summary": "2-3 sentence plain-English explanation", "key_indicators": ["...", "..."], "recommended_action": "..." }` 3. The LLM must **never invent numbers** not present in the input — show how your prompt enforces grounding. 4. Handle LLM failures gracefully (retry once, then fall back to a template-based explanation).

Part D — Serving Layer (Day 3, ~4 hrs)

Expose everything through a **FastAPI** service:

Endpoint	Purpose
`POST /score`	Accepts a transaction JSON, returns fraud probability + risk level
`POST /explain`	Accepts a transaction JSON, returns score + LLM explanation (Part C schema)
`GET /model/info`	Returns model version, training date, data hash, seed, and headline metrics
`GET /health`	Basic health check

Requirements: - Pydantic request/response models with validation - Proper error responses (422 for bad input, 503 if model not loaded) - **Bonus:** Dockerfile + docker-compose (API + Postgres)

---

4. Deliverables

Submit a Git repository (GitHub link or zip) containing:

1. `README.md` — setup instructions, architecture diagram (even ASCII is fine), key decisions and trade-offs
2. `notebooks/` — EDA + model comparison notebook(s)
3. `src/` — pipeline, training script, FastAPI app
4. `models/` — saved model artifact + metadata JSON (version, seed, data hash, metrics)
5. `REPORT.md` — max 2 pages:
6. Model comparison table with your chosen metrics
7. Threshold selection rationale (with the cost math)
8. What you would do differently with 2 more weeks
9. Sample API calls (curl commands or a `requests` demo script)

---

5. Evaluation Rubric (100 points)

Area	Points	What we look for
Data handling & EDA	15	Leakage awareness, imbalance understanding, reproducible pipeline
ML modeling	25	Metric choice, threshold logic, cost-based reasoning — not blind accuracy
Reproducibility & audit trail	15	Seeds, versioning, data hashing, retrainable results
LLM integration	20	Prompt design, structured output, grounding, failure handling
API & code quality	15	Clean FastAPI design, validation, error handling, project structure
Documentation & communication	10	Clear README, honest trade-off discussion in REPORT.md

Bonus (+10): Dockerized setup, unit tests, or a simple monitoring hook (e.g., logging prediction distributions for drift detection).

---

6. Rules & Notes

- You may use any open-source libraries. AI coding assistants are allowed — but you must be able to explain every design decision in the review call.
- Do **not** spend time on frontend/UI. This is a backend + ML assignment.
- Partial submissions are acceptable — a well-reasoned, incomplete solution beats a complete but sloppy one.
- If anything is ambiguous, make a reasonable assumption and **document it** in the README.

Submission deadline: 3 calendar days from receipt. **Review:** 45-minute walkthrough call where you demo the API and defend your modeling choices.